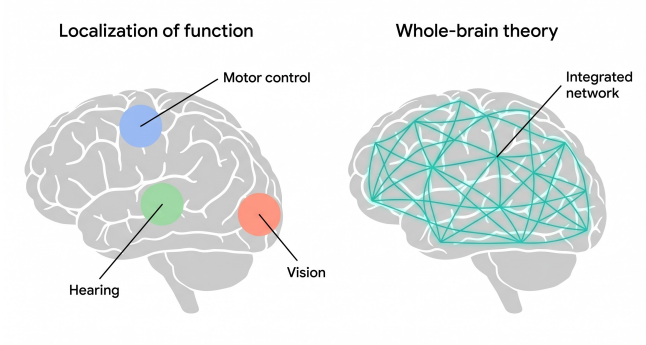
Métricas de Conectividad Funcional Cerebral

https://gemini.google.com/app/1c362c6e80763304

User prompt: Dame unos datos para generar métricas de conectividad funcional

Response: Para estructurar y generar métricas robustas de **conectividad funcional** a partir de datos de resonancia magnética funcional en estado de reposo (**rs-fMRI**), el flujo analítico parte de series temporales regionales hasta llegar a la caracterización topológica mediante teoría de grafos.



1. Datos de Entrada: Matriz de Series Temporales ($T \times N$)

El punto de partida fundamental es extraer la señal BOLD (Blood-Oxygen-Level Dependent) preprocesada (corrección de movimiento, filtrado pasa-banda ej. $0.01 - 0.1$ Hz, regresión de ruidos de sustancia blanca y líquido cefalorraquídeo):

- T: Número de volúmenes o puntos temporales (tr, time points).
- N: Número de nodos o regiones de interés (ROIs) definidas por un atlas anatómico o funcional (por ejemplo, el atlas de Schaefer de 200 o 400 parcelas, o AAL).
- **Estructura:** Una matriz numérica de tamaño $T \times N$ (en Python/NumPy: `shape = (T, N)`).

2. Matriz de Conectividad Funcional ($N \times N$)

A partir de las series temporales, se calcula la interdependencia estadística entre cada par de regiones:

- **Correlación de Pearson:** Se obtiene la matriz de covarianza normalizada de tamaño $N \times N$:

$$R_{ij} = \text{corr}(x_i, x_j)$$

- **Transformación z de Fisher:** Para estabilizar la varianza antes de análisis estadísticos poblacionales:

$$Z_{ij} = 0.5 \ln \left(\frac{1 + R_{ij}}{1 - R_{ij}} \right)$$

- **Correlación Parcial:** Útil para eliminar efectos indirectos y aislar conexiones directas entre nodos (habitualmente regularizada con la inversa de la matriz de covarianza / precisión).

3. Métricas Topológicas (Teoría de Grafos)

Una vez binarizada o ponderada la matriz Z aplicando un umbral de densidad de aristas (sparsity threshold, ej. $10\% - 20\%$), se extraen métricas clave para evaluar la eficiencia de la red y la segregación/integración cerebral:

Categoría	Métrica	Descripción e Interpretación
Segregación	**Coeficiente de Agrupamiento (C / *Clustering*)**	Mide la probabilidad de que los vecinos de un nodo estén conectados entre sí. Refleja la capacidad de procesamiento local especializado.
Segregación	**Modularidad (Q)**	Cuantifica qué tan bien se puede subdividir la red en módulos o comunidades densamente conectadas internamente (ej. Red de Modo Default, Red Frontoparietal).
Integración	**Camino Mínimo Promedio (L / *Characteristic Path Length*)**	Promedio de las distancias más cortas entre todos los pares de nodos. Valores bajos indican alta capacidad de comunicación global.
Integración	**Eficiencia Global y Local**	Eficiencia en la transferencia de información a nivel de toda la red (global) o dentro de vecindarios subgráficos (local).
Centralidad	**Centralidad de Intermediación (*Betweenness Centrality*)**	Identifica los "nodos puente" o *hubs* críticos que participan en el mayor número de caminos más cortos entre otros pares de regiones.

Ejemplo de Implementación en Python (NumPy & NetworkX)

Para generar la matriz de correlación y extraer métricas básicas de red de forma programática:

```
import networkx as nx
import numpy as np

# Simulación de datos BOLD: T=200 volúmenes, N=100 regiones (ROIs)
np.random.seed(42)
bold_signals = np.random.randn(200, 100)

# 1. Matriz de Conectividad Funcional (Correlación de Pearson)
fc_matrix = np.corrcoef(bold_signals.T)

# Rellenar la diagonal con ceros para evitar auto-conexiones
np.fill_diagonal(fc_matrix, 0)

# 2. Umbralización para crear un grafo disperso (ej. conservar el top 15% de conexiones)
threshold = np.percentile(fc_matrix, 85)
adjacency_matrix = np.where(fc_matrix > threshold, fc_matrix, 0)

# 3. Construcción del grafo con NetworkX
G = nx.from_numpy_array(adjacency_matrix)

# 4. Cálculo de métricas clave
clustering_coef = nx.average_clustering(G, weight="weight")
try:
  path_length = nx.average_shortest_path_length(G, weight="weight")
except nx.NetworkXError:
  path_length = np.inf  # Si el grafo no es totalmente conexo

print(f"Coeficiente de agrupamiento promedio: {clustering_coef:.4f}")
print(f"Longitud de camino característico: {path_length}")
```

User prompt: Si

Response: Para evaluar la **eficiencia metabólica cerebral** y su relación con la organización topológica de las redes funcionales mediante rs-fMRI, la teoría de grafos permite cuantificar el equilibrio crítico entre el costo energético (cableado axonal/metabólico) y la capacidad computacional (segregación vs. integración).

1. Modularidad (Q) y Segregación Funcional

La **modularidad** mide qué tan bien se puede subdividir una red cerebral en comunidades (módulos) densamente interconectadas internamente (como la red visual, somatomotora o de modo default) con pocas conexiones entre dichos módulos.

- **Significado Metabólico:** Los módulos densos minimizan la longitud de las rutas de cableado metabólicamente costosas de larga distancia. Un cerebro altamente modular procesa información localmente de manera eficiente y con bajo coste de energía basal.
- **Cálculo:** Se optimiza mediante algoritmos (como Louvain) que maximizan la densidad de enlaces intra-comunidad frente a un modelo nulo:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

Donde A_{ij} es la matriz de adyacencia, k_i el grado del nodo i, m el total de pesos de la red, y $\delta(c_i, c_j)$ indica si los nodos pertenecen al mismo módulo.

2. Eficiencia Global y Eficiencia Local

Mientras que el camino mínimo (L) puede verse distorsionado por desconexiones, la **eficiencia** utiliza el inverso de las distancias nodales, lo que permite manejar redes con componentes disconexos:

- **Eficiencia Global (E_{global}):** Promedio de la inversa de la distancia más corta entre todos los pares de nodos en la red.

$$E_{global} = \frac{1}{N(N-1)} \sum_{i \neq j} \frac{1}{d_{ij}}$$

Interpretación: Capacidad de transferencia de información en paralelo a nivel de todo el cerebro. Requiere mayor consumo energético debido al mantenimiento de conexiones de largo alcance.

- **Eficiencia Local (E_{local}):** Eficiencia global calculada exclusivamente sobre el subgrafo vecindario de cada nodo (excluyendo al nodo mismo).

Interpretación: Refleja la tolerancia a fallos y el procesamiento especializado a escala de vecindario, con un costo metabólico local optimizado.

3. Implementación en Python (NetworkX) para Modularidad y Eficiencia

El siguiente script calcula la modularidad (usando la partición de Louvain) y las eficiencias global y local a partir de la matriz de adyacencia:

```
import community as community_louvain  # pip install python-louvain
import networkx as nx
import numpy as np

# 1. Simular matriz de adyacencia (grafo ponderado y disperso)
np.random.seed(42)
N = 100
random_mat = np.random.rand(N, N)
# Simular simetría y matriz de adyacencia umbralizada
adj = (random_mat + random_mat.T) / 2
np.fill_diagonal(adj, 0)
threshold = np.percentile(adj, 85)
adj_sparse = np.where(adj > threshold, adj, 0)

# Construir grafo ponderado
G = nx.from_numpy_array(adj_sparse)

# 2. Cálculo de la Modularidad (Q) usando el algoritmo de Louvain
# Nota: requiere que los pesos se traten adecuadamente (ej. weights como conductancias)
partition = community_louvain.best_partition(G, weight="weight")
modularity_q = community_louvain.modularity(partition, G, weight="weight")

# 3. Cálculo de Eficiencia Global y Local (en grafos ponderados)
global_eff = nx.global_efficiency(G)
local_eff = nx.local_efficiency(G)

print(f"Índice de Modularidad (Q): {modularity_q:.4f}")
print(f"Eficiencia Global: {global_eff:.4f}")
print(f"Eficiencia Local: {local_eff:.4f}")
```

4. Relación con la Eficiencia Metabólica Cerebral

- **Economía de Conectividad:** Un cerebro sano mantiene un régimen de **"mundo pequeño" (*small-world*)**, combinando una alta eficiencia global (para integrar información cognitiva compleja) con una alta eficiencia local y modularidad (para mantener la segregación de tareas específicas).
- **Costo-Eficiencia:** Desviaciones hacia una aleatoriedad excesiva incrementan drásticamente el consumo metabólico de glucosa y oxígeno al sobrecargar las conexiones de larga distancia. Por el contrario, una hiper-segregación reduce la flexibilidad cognitiva.

User prompt: Si

Response: Para extraer métricas de conectividad funcional directamente desde imágenes Nifti de rs-fMRI, el estándar en el ecosistema de neuroimagen en Python es utilizar **Nilearn** para la extracción de las series temporales mediante un atlas anatómico o funcional (por ejemplo, el atlas de Schaefer) y **NetworkX** junto con librerías de grafos para la topología.

A continuación, tienes un pipeline completo y reproducible que puedes adaptar para procesar tus datos de 3T rs-fMRI:

Pipeline Completo en Python (Nilearn + NetworkX)

```
import community as community_louvain
import networkx as nx
import numpy as np
from nilearn import datasets
from nilearn.input_data import NiftiLabelsMasker  # o NiftiMapsMasker para probabilísticos
from nilearn.connectome import ConnectivityMeasure

# ========================================
# 1. Cargar Datos y Atlas de Referencia
# ========================================
# Para este ejemplo, descargamos un dataset de juguete de Nilearn y el atlas de Schaefer 2018 (400 regiones)
data = datasets.fetch_development_fmri(n_subjects=1)
func_file = data.func[0]  # Ruta al archivo Nifti 4D de rs-fMRI preprocesado

# Cargar el atlas de Schaefer de 400 parcelas y 7 redes
schaefer = datasets.fetch_atlas_schaefer_2018(n_rois=400, yeo_networks=7)
atlas_filename = schaefer.maps
```

```
print(f"Archivo funcional: {func_file}")
print(f"Atlas cargado: {atlas_filename}")

# ========================================
# 2. Extracción de Series Temporales (Masker)
# ========================================
# Configuramos el masker para extraer la señal BOLD promedio de cada ROI del atlas
masker = NiftiLabelsMasker(
    labels_img=atlas_filename,
    standardize="zscore_sample",  # Estandarización temporal (media 0, varianza 1)
    detrend=True,
    low_pass=0.08,
    high_pass=0.009,
    t_r=2.0,  # TR de la adquisición (ajustar según tu escáner 3T)
    memory="nilearn_cache",
    verbose=0,
)

# Extraer matriz de series temporales (T x N: Volúmenes x Regiones)
time_series = masker.fit_transform(func_file)
print(f"Dimensiones de la matriz de series temporales (T x N): {time_series.shape}")

# ========================================
# 3. Cálculo de la Matriz de Conectividad
# ========================================
# Nilearn facilita el cálculo de correlación de Pearson y la transformación Fisher-z
correlation_measure = ConnectivityMeasure(
    kind="correlation", vectorize=False, discard_diagonal=True
)
fc_matrix = correlation_measure.fit_transform([time_series])[
    0
]  # Matriz N x N

# Aplicar transformación Fisher z opcional para análisis estadísticos
fc_z_matrix = np.arctanh(
    np.clip(fc_matrix, -0.999, 0.999)
)  # Evitar infinitos
np.fill_diagonal(fc_z_matrix, 0)

# ========================================
# 4. Construcción del Grafo y Umbralización
# ========================================
# Aplicar un umbral de densidad (ej. conservar el top 15% de conexiones más fuertes)
sparsity = 0.15
flat_weights = fc_z_matrix[np.triu_indices_from(fc_z_matrix, k=1)]
threshold = np.percentile(flat_weights, 100 * (1 - sparsity))

adjacency_matrix = np.where(fc_z_matrix > threshold, fc_z_matrix, 0)

# Crear grafo ponderado dirigido/no dirigido con NetworkX
G = nx.from_numpy_array(adjacency_matrix)

# ========================================
# 5. Extracción de Métricas Topológicas
# ========================================
# A. Modularidad (Q) mediante algoritmo de Louvain
partition = community_louvain.best_partition(G, weight="weight")
modularity_q = community_louvain.modularity(partition, G, weight="weight")

# B. Eficiencias Global y Local
global_eff = nx.global_efficiency(G)
local_eff = nx.local_efficiency(G)

# C. Coeficiente de Agrupamiento y Longitud de Camino Característico
clustering_coef = nx.average_clustering(G, weight="weight")
try:
  path_length = nx.average_shortest_path_length(G, weight="weight")
except nx.NetworkXError:
  path_length = np.nan  # Manejo por si el grafo queda disconexo

print("\n--- Resultados de Métricas de Conectividad ---")
```

```
print(f"Modularidad (Q): {modularity_q:.4f}")
print(f"Eficiencia Global: {global_eff:.4f}")
print(f"Eficiencia Local: {local_eff:.4f}")
print(f"Coeficiente de Agrupamiento: {clustering_coef:.4f}")
print(f"Camino Mínimo Promedio: {path_length}")
```

Consideraciones Clave para un Entorno de Alto Rendimiento (HPC / Cluster)

1. **Control de Conectividad Espuria (Spurious Correlations):** Asegúrate de que las imágenes Nifti de entrada ya incluyan la regresión de señales de ruido (bandas de desalineación de movimiento 24 parámetros, materia blanca, LCF) antes de pasarlas por el `NiftiLabelsMasker`.
2. **Estrategia de Umbralización:** En estudios de neuroimagen poblacionales, suele preferirse fijar la **densidad de aristas proporcional** (ej. mismo número de conexiones para todos los sujetos) en lugar de un umbral de correlación absoluto, para evitar sesgos de grado globales asociados al tamaño del componente conexo.

User prompt: Si

Response: Para procesar grandes cohortes de imágenes rs-fMRI de forma eficiente y aprovechar al máximo los recursos de cálculo (como un nodo multihilo de un clúster HPC o una estación de trabajo de alta gama), la mejor opción en Python es utilizar **`joblib`** junto con una función contenedora robusta.

Esto permite paralelizar el procesamiento por sujeto, aislar errores (para que si un sujeto falla, no se detenga toda la ejecución) y consolidar automáticamente los resultados en una tabla tabular (`pandas.DataFrame`).

Script Optimizado de Procesamiento en Paralelo (`joblib`)

```
import os
import community as community_louvain
import joblib
from joblib import Parallel, delayed
import networkx as nx
import numpy as np
from nilearn.connectome import ConnectivityMeasure
from nilearn.input_data import NiftiLabelsMasker
import pandas as pd


def process_single_subject(func_file, atlas_filename, sparsity=0.15, tr=2.0):
  """Procesa un único sujeto: extracción BOLD, matriz de conectividad y grafos."""
  try:
    # 1. Configurar Masker local para el sujeto
    masker = NiftiLabelsMasker(
        labels_img=atlas_filename,
        standardize="zscore_sample",
        detrend=True,
        low_pass=0.08,
        high_pass=0.009,
        t_r=tr,
        memory="nilearn_cache",  # Caché en disco para acelerar operaciones repetidas
        verbose=0,
    )
    time_series = masker.fit_transform(func_file)

    # 2. Matriz de Conectividad Funcional
    correlation_measure = ConnectivityMeasure(
        kind="correlation", vectorize=False, discard_diagonal=True
    )
    fc_matrix = correlation_measure.fit_transform([time_series])[0]

    # Transformación Fisher-z y limpieza de diagonal
    fc_z = np.arctanh(np.clip(fc_matrix, -0.999, 0.999))
    np.fill_diagonal(fc_z, 0)

    # 3. Umbralización basada en densidad proporcional (Sparsity)
    flat_weights = fc_z[np.triu_indices_from(fc_z, k=1)]
    threshold = np.percentile(flat_weights, 100 * (1 - sparsity))
    adjacency_matrix = np.where(fc_z > threshold, fc_z, 0)

    # 4. Construcción del Grafo
    G = nx.from_numpy_array(adjacency_matrix)
```

```
    # 5. Extracción de Métricas Topológicas
    partition = community_louvain.best_partition(G, weight="weight")
    mod_q = community_louvain.modularity(partition, G, weight="weight")
    global_eff = nx.global_efficiency(G)
    local_eff = nx.local_efficiency(G)
    clustering = nx.average_clustering(G, weight="weight")

    try:
      path_len = nx.average_shortest_path_length(G, weight="weight")
    except nx.NetworkXError:
      path_len = np.nan  # Grafo disconexo

    return {
        "subject_id": os.path.basename(func_file).split("_")[0],
        "modularity_Q": mod_q,
        "global_efficiency": global_eff,
        "local_efficiency": local_eff,
        "clustering_coefficient": clustering,
        "characteristic_path_length": path_len,
        "status": "success",
    }

  except Exception as e:
    # Capturar errores por sujeto sin romper el pipeline completo
    sub_id = (
        os.path.basename(func_file).split("_")[0]
        if "func_file" in locals()
        else "unknown"
    )
    return {"subject_id": sub_id, "status": f"error: {str(e)}"}


# ==========================================
# Ejecución Principal por Lotes (Batch)
# ==========================================
if __name__ == "__main__":
  # Lista simulada de rutas a archivos Nifti 4D preprocesados (BIDS layout o plano)
  subject_files = [
      "/ruta/dataset/sub-01/func/sub-01_task-rest_bold.nii.gz",
      "/ruta/dataset/sub-02/func/sub-02_task-rest_bold.nii.gz",
      "/ruta/dataset/sub-03/func/sub-03_task-rest_bold.nii.gz",
      # ... añade todos los sujetos de tu cohorte aquí
  ]

  atlas_path = "/ruta/atlas/schaefer2018_400_7networks.nii.gz"

  print(f"Iniciando procesamiento paralelo para {len(subject_files)} sujetos...")

  # n_jobs=-2 utiliza todos los núcleos disponibles de la CPU menos uno (ideal para no congelar el sistema)
  # verbose=10 muestra una barra de progreso detallada en consola
  results = Parallel(n_jobs=-2, verbose=10)(
      delayed(process_single_subject)(f_file, atlas_path, sparsity=0.15, tr=2.0)
      for f_file in subject_files
  )

  # Consolidar resultados en un DataFrame de Pandas
  df_cohort = pd.DataFrame(results)

  # Filtrar y reportar errores si los hubo
  failed_subjects = df_cohort[df_cohort["status"] != "success"]
  if not failed_subjects.empty:
    print(
        f"\n[Advertencia] {len(failed_subjects)} sujetos fallaron en el"
        " procesamiento:"
    )
    print(failed_subjects[["subject_id", "status"]])

  # Guardar la tabla final para análisis estadístico (ej. correlaciones clínicas o machine learning)
  output_csv = "functional_connectivity_metrics_cohort.csv"
  df_cohort.to_csv(output_csv, index=False)
  print(f"\n¡Proceso completado! Métricas guardadas en: {output_csv}")
  print(df_cohort.head())
```

Consideraciones Clave para Rendimiento en Entornos de Alta Carga:

1. **Gestión de Memoria y Multihilo:** Nilearn y NumPy optimizan internamente las operaciones de álgebra lineal utilizando bibliotecas como OpenBLAS o MKL. Si lanzas `joblib` con muchos hilos (`n_jobs`), asegúrate de limitar los hilos internos de BLAS (`export OMP_NUM_THREADS=4`) para evitar la saturación de contexto por *oversubscription* en la CPU.
2. **Caché de Nilearn (`memory="nilearn_cache"`):** Al procesar múltiples sujetos, el atlas de Schaefer o las máscaras se leen repetidamente; el parámetro de caché en disco evita redundancias de lectura de I/O en almacenamiento compartido del clúster.

User prompt: Dame algunas bases de datos para practicar estos scripts

Response: Para poner a prueba y practicar los scripts de extracción de conectividad funcional y teoría de grafos, existen excelentes opciones públicas que van desde datasets pequeños integrados directamente en Python hasta repositorios internacionales completos en formato BIDS.

1. Datasets Integrados en Nilearn (Listos para Descargar con 1 Línea)

Ideales para validar código rápidamente sin lidiar con estructuras de directorios complejas:

- **`fetch_development_fmri` (Desarrollo y Cognición):**
  - *Qué contiene:* Datos rs-fMRI de niños y adolescentes.
  - *Uso en código:* `from nilearn.datasets import fetch_development_fmri; data = fetch_development_fmri(n_subjects=3)`
- **`fetch_adhd` (TDAH - Trastorno por Déficit de Atención e Hiperactividad):**
  - *Qué contiene:* Subconjunto de sujetos con TDAH y controles sanos (incluye datos funcionales y de máscara).
  - *Uso en código:* `from nilearn.datasets import fetch_adhd; data = fetch_adhd(n_subjects=10)`

2. ABIDE (Autism Brain Imaging Data Exchange)

- **Descripción:** Es uno de los datasets clínicos más utilizados en machine learning para neuroimagen. Contiene datos de rs-fMRI e información fenotípica (controles sanos vs. trastornos del espectro autista).
- **Cómo obtenerlo:**
  - Puedes descargar subconjuntos preprocesados mediante el pipeline CPAC a través de la API de Nilearn (`nilearn.datasets.fetch_abide_pcp`).
  - O desde su portal oficial en el consorcio NITRC / ABIDE Home.

3. OpenNeuro.org (El Estándar BIDS Abierto)

- **Descripción:** Un repositorio masivo que alberga miles de datasets de neuroimagen completamente estructurados bajo el estándar **BIDS** (*Brain Imaging Data Structure*), lo que garantiza que las rutas de archivos (`/func/`, `/anat/`) coincidan exactamente con la estructura que esperan herramientas como Nilearn.
- **Datasets recomendados para empezar con rs-fMRI:**
  - **`ds000114`:** Datos multimodales de reposo y tareas.
  - **`ds000228`:** Datos de resonancia magnética funcional en estado de reposo para estudios longitudinales o de cohorte.
- **Cómo descargarlos:** Se pueden clonar fácilmente mediante **DataLad** o descargar carpetas específicas directamente desde el navegador o vía AWS/GCP S3 sin costo.

4. Human Connectome Project (HCP - S1200 / Minirelease)

- **Descripción:** El estándar de oro en neuroimagen de alta resolución (3T y 7T con tiempos de repetición cortos y alta resolución espacial).
- **Qué contiene:** Series temporales de rs-fMRI limpias y de alta calidad junto con matrices de conectividad ya calculadas (en algunas versiones).
- **Cómo obtenerlo:** Requiere crear una cuenta gratuita en ConnectomeDB para aceptar los términos de uso de datos abiertos de la NIMH.
